Reconocimiento de Patrones — Examen Intermedio (Examen B)

Universidad de Sonora · Licenciatura en Ciencias de la Computación

Alumno: Owen Adiel Solis Zatarain
Expediente: 223201053

Semestre 2026-2

Parámetros personales del examen:	
Expediente completo (semilla):	`223201053`
Último dígito (a):	$a = 3$
Penúltimo dígito (b):	$b = 5$

1. Problema 1: El reporte demasiado bueno

1.1. Parte teórica

(a) Copias en la prueba (6 pts)

1. **Fracción de renglones positivos después de repetir:**

   En el conjunto de datos original existen $N_{\text{neg}} = 39{,}922$ llamadas sin contratación (clase negativa) y $N_{\text{pos}} = 5{,}289$ llamadas con contratación (clase positiva). En el código de la consultora se ejecuta `pd.concat([X] + [X[pos]] * 6)`, lo que concatena la tabla original (que ya contiene una vez a los positivos) con 6 copias adicionales de los casos positivos. En consecuencia, cada renglón positivo aparece 7 veces en total:

   $$N_{\text{pos, nuevo}} = 7 \times 5{,}289 = 37{,}023, \qquad N_{\text{total}} = 39{,}922 + 37{,}023 = 76{,}945$$

   La nueva fracción de renglones positivos en la tabla modificada es:

   $$f_+ = \frac{N_{\text{pos, nuevo}}}{N_{\text{total}}} = \frac{37{,}023}{76{,}945} \approx 0.48116 \quad (48.12\,\%)$$

   Esto distorsiona la distribución real del problema (donde originalmente solo el 11.70 % contrató), haciendo que casi la mitad del conjunto de datos parezca corresponder a contrataciones exitosas.

2. **Probabilidad de que un positivo en prueba tenga al menos una copia en entrenamiento:**

   Si tomamos un renglón positivo que cayó en el conjunto de prueba, sabemos que en el conjunto de datos inflado existen otras 6 copias idénticas de ese mismo cliente. Bajo el supuesto de que cada copia se asigna de forma independiente a entrenamiento con probabilidad 0.8 (80 %) o a

prueba con probabilidad 0.2 (20 %), el único escenario en el que dicho renglón de prueba *no* tiene ninguna copia idéntica en el entrenamiento es que las otras 6 copias hayan caído todas también en el conjunto de prueba:

$$P(\text{ninguna copia en ent.}) = (0.2)^6 = 0.000064$$

Por complemento, la probabilidad de que tenga al menos una copia idéntica en el conjunto de entrenamiento es:

$$q = P(\text{al menos una copia en ent.}) = 1 - (0.2)^6 = 0.999936 \quad (99.9936\,\%)$$

Es decir, prácticamente la totalidad de los casos positivos evaluados en la prueba ya fueron vistos de manera idéntica durante el entrenamiento.

3. **Exactitud medida por la consultora como función de p:**

   Podemos particionar el conjunto de prueba en dos grupos disjuntos:

   - Los renglones positivos que tienen al menos una copia idéntica en el entrenamiento, los cuales representan una fracción $c = f_+ \cdot q$ del conjunto de prueba. Como el modelo memoriza estos casos vistos en entrenamiento, acierta siempre en ellos (exactitud igual a 1).
   - El resto de los renglones del conjunto de prueba (todos los negativos más la fracción minúscula de positivos sin copia en entrenamiento), que representan una proporción $1-c$, donde el modelo tiene su exactitud real p.

   Calculando la proporción c:

$$c = f_+ \cdot q = \frac{37{,}023}{76{,}945} \times 0.999936 \approx 0.48116 \times 0.999936 \approx 0.4811$$

   Por la ley de probabilidad total, la exactitud que mide la consultora en función de p es:

$$\text{Exactitud}(p) = c \cdot (1) + (1 - c) \cdot p \approx 0.4811 + 0.5189\,p$$

   Este resultado demuestra que el código de la consultora tiene garantizado un piso de 48.11 % de exactitud únicamente por memorizar copias repetidas: incluso si el modelo tuviera una exactitud real mediocre o equivalente a lanzar una moneda ($p = 0.5$), el reporte mediría una exactitud inflada de $0.4811 + 0.5189(0.5) \approx 74.06\,\%$.

(b) Elegir con la prueba (5 pts)

Sean $\hat{p}_1, \ldots, \hat{p}_m$ las exactitudes medidas en los n datos de prueba de los m modelos independientes, todos con la misma exactitud real p. Por la propiedad **(P2)**, cada $\hat{p}_j$ sigue aproximadamente una distribución normal con media p y desviación estándar $\sigma = \sqrt{p(1-p)/n}$, por lo que podemos escribirla en términos de una variable aleatoria normal estándar $Z_j \sim \mathcal{N}(0,1)$ independiente como:

$$\hat{p}_j = p + \sqrt{\frac{p(1-p)}{n}}\, Z_j$$

Al elegir el modelo con mayor exactitud en el conjunto de prueba, dado que p y $\sqrt{p(1-p)/n}$ son constantes positivas que no dependen de j, se tiene que:

$$\max_{1 \le j \le m} \hat{p}_j = p + \sqrt{\frac{p(1-p)}{n}} \left(\max_{1 \le j \le m} Z_j \right)$$

Tomando el valor esperado en ambos lados por linealidad de la esperanza y aplicando la propiedad **(P1)** ($\mathbb{E}[\max_j Z_j] \approx \sqrt{2\ln m}$):

$$\mathbb{E}\left[\max_{1\le j\le m} \hat{p}_j\right] = p + \sqrt{\frac{p(1-p)}{n}}\,\mathbb{E}\left[\max_{1\le j\le m} Z_j\right] \approx p + \sqrt{\frac{p(1-p)}{n}}\sqrt{2\ln m}$$

Por lo tanto, el sesgo o sobreestimación esperada respecto a la exactitud real p es:

$$\text{Sesgo} = \mathbb{E}\left[\max_{1\le j\le m} \hat{p}_j\right] - p \approx \sqrt{\frac{p(1-p)}{n}}\sqrt{2\ln m}$$

Sustituyendo los valores numéricos con $p = 0.8$ (de modo que $p(1-p) = 0.8 \times 0.2 = 0.16$):

- **Caso de la consultora ($m = 4$ profundidades probadas, $n = 0.2 \times 76{,}945 = 15{,}389$ datos de prueba):**

$$\text{Sesgo}_{\text{consultora}} \approx \sqrt{\frac{0.8(0.2)}{15{,}389}}\sqrt{2\ln(4)} \approx 0.003224 \times 1.665109 \approx 0.0054 \quad (0.54\,\%)$$

- **Caso personal ($a = 3, b = 5 \implies m = 10^{1+(3 \bmod 3)} = 10^1 = 10$ modelos, $n = 100(5+1) = 600$ datos de prueba):**

$$\text{Sesgo}_{\text{personal}} \approx \sqrt{\frac{0.8(0.2)}{600}}\sqrt{2\ln(10)} \approx 0.016330 \times 2.145966 \approx 0.0350 \quad (3.50\,\%)$$

¿Cuándo importa este problema y cuándo no? A partir de la expresión $\sqrt{p(1-p)/n}\sqrt{2\ln m}$, observamos que la sobreestimación disminuye con la raíz del tamaño de la muestra de prueba ($1/\sqrt{n}$) y crece lentamente con el número de modelos evaluados ($\sqrt{2\ln m}$). Por ello, este problema **importa mucho** cuando el conjunto de prueba n es pequeño (como en el caso personal con $n = 600$, donde infla la exactitud en 3.50 puntos porcentuales) y cuando se prueban decenas o cientos de combinaciones de hiperparámetros m. En cambio, **casi no tiene impacto numérico** cuando n es muy grande y se comparan pocos modelos (como en el código de la consultora con $n = 15{,}389$ y $m = 4$, donde apenas aporta 0.54 %), aunque metodológicamente sigue siendo una mala práctica.

(c) Seleccionar atributos con ruido (6 pts)

Supongamos que se tienen $n = 50$ datos y $m = 5{,}000$ atributos que son puro ruido, independientes de la clase. Por la propiedad **(P3)**, la correlación muestral r_j entre cada atributo j y la clase sigue aproximadamente una distribución normal con media 0 y desviación estándar $1/\sqrt{n}$, por lo que podemos expresarla como $r_j = \frac{1}{\sqrt{n}} Z_j$, con $Z_j \sim \mathcal{N}(0,1)$. Aplicando la propiedad **(P1)** para estimar el máximo entre los $m = 5{,}000$ atributos:

$$\mathbb{E}\left[\max_{1\le j\le 5000} r_j\right] \approx \frac{1}{\sqrt{50}}\sqrt{2\ln(5{,}000)} \approx 0.141421 \times 4.127273 \approx 0.5837$$

(Si consideramos ambos extremos de la distribución por el valor absoluto $|r_j|$, equivalente a $2m = 10{,}000$ colas, la correlación máxima esperada sube a $\frac{1}{\sqrt{50}}\sqrt{2\ln(10{,}000)} \approx 0.6070$). Es decir, aunque todos los atributos sean ruido puro, por simple azar muestral al tener pocos renglones ($n = 50$) y miles de columnas (5,000), encontraremos atributos con correlaciones aparentes cercanas a 0.60.

¿Por qué infla la validación cruzada elegir antes? Si seleccionamos los 20 atributos más correlacionados utilizando *todos* los 50 datos antes de partir, estamos filtrando aquellas columnas de ruido que, por casualidad, coincidieron con las etiquetas y de esos 50 renglones específicos. Cuando posteriormente dividimos esos mismos 50 renglones en pliegues de entrenamiento y validación cruzada, los datos del pliegue de validación ya intervinieron en la elección de los 20 atributos. El clasificador aprende esa correlación espuria que ya está presente en toda la muestra y obtiene una exactitud muy superior al 50 %, a pesar de que ante datos nuevos su desempeño real sería de 50 %.

Forma correcta de hacerlo: La selección de atributos debe realizarse *dentro* de cada pliegue de la validación cruzada, ajustando el selector (`SelectKBest`) únicamente con los datos de entrenamiento de ese pliegue y aplicándolo después al pliegue de validación (en `scikit-learn`, encapsulando el selector y el clasificador dentro de un `Pipeline`).

(d) ¿Es trampa la duración? (4 pts)

Definición de fuga de información temporal: Sea t_0 el instante en el que se ejecuta el modelo en producción para tomar una decisión sobre una observación. Decimos que una variable X_k presenta *fuga de información* respecto a t_0 si su valor solo se genera, determina u observa en un instante $t > t_0$ (posterior a la toma de decisión o como consecuencia del evento objetivo Y).

¿Es `duration` una fuga de información? Depende de la etapa operativa en la que se utilice el modelo:

- **Uso pre-llamada (t_0 = antes de marcar al cliente): Sí es fuga.** Si el objetivo del banco es decidir a qué clientes de su cartera vale la pena llamar en la campaña, en el instante t_0 la llamada aún no ha ocurrido y es imposible conocer cuántos segundos durará. Además, en la práctica de un centro de llamadas, aunque una llamada larga no garantiza por sí sola que el cliente contrate (una persona puede hacer preguntas durante mucho tiempo y no comprar), una llamada muy corta (por ejemplo, menor al tiempo mínimo indispensable para explicar el producto y tomar los datos del contrato, que suele ser de al menos un minuto) descarta casi con certeza la contratación. Incluir `duration` le permite al modelo descartar trivialmente todas las llamadas cortas usando información del futuro.

- **Uso post-llamada (t_0 = después de finalizar la llamada): No es fuga.** Si el modelo se emplea en una segunda etapa operativa después de haber realizado el contacto telefónico, `duration` ya es un dato conocido ($t \leq t_0$). Por ejemplo, se puede ordenar el histórico de llamadas exitosas para establecer el tiempo mínimo requerido para concretar una contratación: si el cliente no contesta (0 s), se mantiene como potencial para reintentar después; si contesta pero cuelga por debajo de ese umbral mínimo, se descarta; y si supera el umbral sin haber cerrado la contratación en ese instante, se prioriza para seguimiento o auditoría de calidad.

¿Qué otras variables tienen el mismo problema? En el escenario pre-llamada (antes de iniciar los contactos de la campaña para un cliente), presentan el mismo problema las variables registradas al cierre del contacto de la campaña actual:

- `campaign`: representa el número total de llamadas realizadas al cliente durante esta campaña (incluyendo el último contacto). Antes de empezar a llamarle, es imposible saber cuántos intentos tomará localizarlo o convencerlo.

- `day` y `month`: registran el día y el mes en que ocurrió el *último* contacto de la campaña, fecha que tampoco se conoce antes de efectuar las llamadas.

(e) El tiempo (4 pts)

Por qué falla una partición aleatoria: Las 45,211 llamadas se realizaron entre mayo de 2008 y noviembre de 2010, un periodo marcado por fuertes cambios macroeconómicos (la crisis financiera de 2008 frente a la estabilización posterior). Una partición aleatoria mezcla llamadas de todas las fechas tanto en entrenamiento como en prueba, lo cual distorsiona la estimación en dos sentidos:

- **Puede ser demasiado optimista:** porque para predecir el comportamiento de clientes en un mes determinado (por ejemplo, mediados de 2010), el modelo ya fue entrenado con llamadas ocurridas en ese mismo mes y bajo las mismas condiciones económicas, algo imposible en la vida real donde el futuro aún no ocurre.
- **Puede ser pesimista o contaminar el modelo:** porque los meses más críticos de 2008 (cuando la economía estaba en crisis y el desempeño de las campañas era anormalmente bajo) se mezclan con los datos recientes, contaminando tanto el aprendizaje de las tendencias actuales como la evaluación de lo que ocurrirá en la siguiente campaña.

Partición propuesta que imita el uso real: Para evaluar honestamente cómo funcionará el modelo en la próxima campaña, debemos respetar el orden cronológico:

1. **Corte cronológico para prueba:** Reservar el último bloque temporal de los datos (por ejemplo, el último 20 % de los renglones, correspondiente a los meses más recientes de 2010) como conjunto de prueba final.
2. **Validación con ventana deslizante (*sliding window*):** En lugar de acumular todos los datos desde mayo de 2008, entrenar y validar avanzando por bloques temporales mediante una ventana móvil de tamaño fijo que utilice únicamente los meses recientes previos al bloque que se desea predecir, descartando así los meses antiguos afectados por eventos económicos atípicos como la crisis de 2008.

1.2. Carta al banco (Inciso f de la parte práctica)

Estimado Banco:

El modelo que les entregó la consultora exageraba lo que pueden esperar de su próxima campaña telefónica. Su 93 % de aciertos no significa mucho: como solo el 12 % de los clientes contrató, un modelo que dijera "no" a todos ya acertaría el 88 %. Revisamos el código y encontramos tres errores principales:

- **Usaba la duración de la llamada.** El modelo usaba cuánto duró cada llamada, pero ese dato solo se conoce *después* de llamar. Es como adivinar quién va a contratar sabiendo ya si la conversación fue larga. Cuando quitamos ese dato, el desempeño cayó notablemente. Lo mismo pasa con el número de llamadas de la campaña y la fecha del último contacto.
- **Mezcló las fechas.** Las llamadas van de 2008 a 2010, y la consultora las revolvió antes de probar el modelo. Así, el modelo aprendía con llamadas de 2010 para "predecir" llamadas de 2008, algo que en la vida real nunca va a pasar: ustedes siempre van a predecir la próxima campaña con información del pasado. Además, el porcentaje de clientes que contrata cambió mucho con el tiempo (en los primeros meses contrataba cerca del 3 % y al final casi la mitad), así que mezclar fechas esconde ese cambio. Este fue el error que más infló el resultado.
- **Calificó al modelo con datos que ya conocía.** Para compensar que pocos clientes contratan, duplicaron seis veces las llamadas con contratación, y lo hicieron antes de separar los datos de entrenamiento y de prueba. También eligieron las variables y el ajuste final mirando los datos de prueba. Es como presentar un examen después de haber visto las respuestas.

¿Qué desempeño es realista? Corregimos todo y probamos el modelo como se va a usar: entrenándolo con las llamadas más antiguas y midiéndolo con las más recientes, que nunca vio. El modelo no sirve para decir con certeza quién va a contratar, pero sí para **ordenar** a sus clientes de mayor a menor probabilidad de contratar. Si llaman primero al 10 % de clientes que el modelo pone hasta arriba, cerca del **70 %** contrata (entre 67 % y 73 %), contra el **32 %** si llaman al azar en ese mismo periodo. Es decir, consiguen alrededor de **2.2 veces más contrataciones por llamada**.

Lo que recomiendo:

1. Usar el modelo para decidir *en qué orden* llamar a sus clientes, empezando por los de mayor probabilidad, y no como una respuesta de "sí" o "no".
2. Actualizar el modelo antes de cada campaña usando solo las llamadas más recientes. Encontramos que entrenar con las llamadas recientes funciona mejor que usar todo el historial, porque el comportamiento de los clientes cambia con el tiempo.
3. Pedirle a cualquier proveedor que demuestre su modelo con campañas *posteriores* a las que usó para construirlo, y desconfiar de resultados casi perfectos.
4. Medir los resultados reales de cada campaña y compararlos con lo esperado, para detectar a tiempo si el modelo deja de funcionar.

Atentamente,
Owen Adiel Solis Zatarain

1.3. Bitácora de decisiones — Problema 1

Para cada decisión anotamos qué decidimos, qué alternativa descartamos, por qué, qué evidencia nos convenció y qué perdemos con ella. Los números vienen del notebook `problema-1/problema_1.ipynb`.

Decisión 1: Qué variables son fuga de información (Tú decides, teoría (d))

Decisión. Usamos el modelo *antes* de llamar al cliente, así que quitamos todas las variables que se registran durante o después del contacto de la campaña actual: `duration`, `campaign`, `day` y `month`.

Alternativa descartada. Quitar solo `duration`, que es la fuga más evidente.

Por qué. Con nuestra definición de fuga (una variable que no se conoce en el momento t_0 en que se usa el modelo), `campaign`, `day` y `month` tienen el mismo problema que `duration`: antes de llamar no se sabe cuántos intentos habrá ni en qué fecha será el último contacto.

Evidencia. En la tabla del inciso (c), quitar `duration` baja el AUC de 0.893 a 0.750, y quitar además las otras tres lo baja a 0.734. Esa diferencia adicional es información que el modelo no tendría en la práctica.

Qué perdemos. El modelo no sirve para el uso posterior a la llamada (por ejemplo, priorizar seguimientos), donde `duration` sí sería válida. Además, `month` podría conocerse si la campaña ya está calendarizada; al quitarla, perdemos la estacionalidad.

Decisión 2: Partición cronológica (Tú decides, práctica (d))

Decisión. El último 20 % de las llamadas es la prueba. Los hiperparámetros se eligen con `TimeSeriesSplit` (5 pliegues) dentro del entrenamiento, así que siempre se entrena con el pasado y se valida con el futuro.

Alternativa descartada. Partición al azar estratificada, con `StratifiedKFold`.

Por qué. El banco va a usar el modelo para la *próxima* campaña, con datos de campañas anteriores. Una partición al azar deja que el modelo vea el futuro, como explicamos en la teoría (e).

Evidencia. Corregir solo la partición baja el AUC de 0.893 a 0.680: es la corrección individual que más pesa. Además, la tasa de contratación por bloques cronológicos sube de 3 % a 47 %, y en la prueba es de 31.6 % contra ≈ 5 % en el entrenamiento. Una partición al azar esconde ese cambio.

Qué perdemos. Los primeros pliegues entrenan con pocos datos y la validación cruzada es muy ruidosa: su AUC es ≈ 0.54 y la desviación entre pliegues es de ≈ 0.1. Por eso, con la validación cruzada es difícil distinguir entre hiperparámetros parecidos.

Decisión 3: Ventana deslizante de entrenamiento

Decisión. Tratamos el tamaño de la ventana como un hiperparámetro más: creciente o las últimas 3,000, 6,000, 12,000 o 18,000 llamadas. La validación cruzada temporal eligió las **últimas 3,000 llamadas**, y el modelo final se entrena solo con ellas.

Alternativa descartada. La ventana creciente, que usa todo el historial desde mayo de 2008.

Por qué. En la teoría (e) propusimos una ventana deslizante porque los meses viejos se parecen poco a la siguiente campaña. Quisimos ponerlo a prueba en lugar de solo afirmarlo.

Evidencia. La validación cruzada apenas distingue entre ventanas (entre 0.092 y 0.119 de precisión en el 10 % superior, con desviación de ≈ 0.1). Como diagnóstico, que no se usó para elegir, comparamos en la prueba: la ventana de 3,000 supera a la creciente por 6.0 puntos de precisión en el 10 % superior, con un intervalo de confianza del 95 % por bootstrap pareado de $[+3.3, +9.1]$ que no incluye el cero.

Qué perdemos. Entrenamos con menos del 10 % de los datos disponibles, así que el modelo tiene más varianza. Además, la relación no es monótona (la ventana de 18,000 quedó peor que la creciente), por lo que el tamaño óptimo puede cambiar de una campaña a otra.

Decisión 4: Tratamiento del desbalance (Tú decides, práctica (d))

Decisión. No repetimos renglones. Comparamos *no hacer nada* contra `class_weight="balanced"` (pesos, sin copias) como un hiperparámetro de la búsqueda. Las dos opciones quedaron empatadas y la búsqueda tomó `balanced` con $C = 1$.

Alternativa descartada. Repetir positivos o usar SMOTE dentro de cada pliegue.

Por qué. El banco va a usar las probabilidades para *ordenar* clientes, no el corte en 0.5. En una regresión logística, los pesos casi solo mueven el intercepto, así que el orden casi no cambia. Repetir renglones agrega el riesgo de que las copias se crucen entre pliegues (el error de la consultora), y SMOTE necesita una librería adicional.

Evidencia. Con la ventana elegida, la mejor combinación de cada opción llega a 0.119 de precisión en el 10 % superior en la validación cruzada. Con la ventana creciente, la diferencia entre las dos opciones fue menor a 0.01, contra una desviación entre pliegues de 0.09.

Qué perdemos. Si el banco quisiera una respuesta de "sí/no" con un umbral fijo, las probabilidades sin balancear casi nunca pasarían de 0.5. Habría que elegir el umbral aparte.

Decisión 5: Métrica de reporte (Tú decides, práctica (d))

Decisión. La métrica principal es la **precisión en el 10 % superior**: si el banco llama al 10 % de clientes con mayor probabilidad, ¿qué proporción contrata? La acompañamos del *lift* sobre la tasa base de la prueba, el AUC-PR y el AUC-ROC, todos con un intervalo de confianza del 95 % por bootstrap de la prueba (1,000 remuestreos).

Alternativa descartada. La exactitud, que reportó la consultora.

Por qué. La métrica tiene que medir cómo se va a usar el modelo: llamar primero a los más probables. La exactitud premia decir "no" a todos.

Evidencia. En el inciso (c), la exactitud trivial (predecir siempre la clase mayoritaria) va de 0.518 a 0.877 según la variante, y la variante sin copias da una exactitud de 0.848, *peor* que la trivial de 0.877. La exactitud engaña. Con nuestra métrica, el modelo final da 0.702 $[0.675, 0.732]$ contra una tasa base de 0.316, es decir, un lift de 2.22.

Qué perdemos. El 10 % es una suposición sobre la capacidad del centro de llamadas. Si el banco llama a otra fracción, hay que recalcular la métrica; la curva de ganancia del notebook muestra todas las fracciones.

Decisión 6: Clasificador del experimento del ruido (práctica (e))

Decisión. Usamos regresión logística, con 100 repeticiones y validación cruzada estratificada de 5 pliegues.

Alternativa descartada. k vecinos más cercanos.

Por qué. Es la misma familia de modelos que en el resto del problema, así que el experimento ilustra directamente el error de `SelectKBest` de la consultora.

Evidencia. Con la selección antes de la validación cruzada, la exactitud estimada es de 99.1 % en promedio; con la selección dentro de cada pliegue, de 49.4 %. Las distribuciones no se traslapan.

Qué perdemos. No comprobamos si un clasificador más flexible exagera aún más. Con 45 atributos y 40 datos de entrenamiento, la regresión logística ya separa casi perfectamente, así que el resultado probablemente no cambiaría.

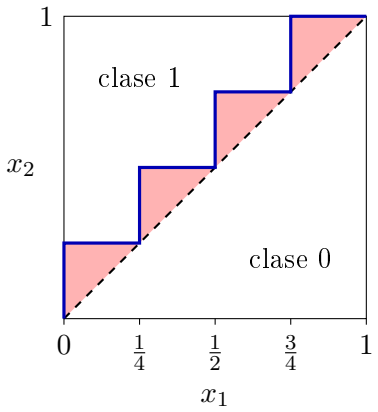
2. Problema 2: Árboles que cortan en diagonal

2.1. Parte teórica

(a) La escalera (5 pts)

Los datos son uniformes en el cuadrado $[0,1]^2$, que tiene área 1, y la clase es 1 si $x_2 > x_1$, es decir, si el punto está arriba de la diagonal. Como los datos son uniformes, la probabilidad de clasificar mal un punto es igual al *área* de la región que el árbol clasifica mal. Por eso basta con calcular áreas.

Cómo es la escalera. Dividimos el eje horizontal en m pedazos de ancho $1/m$, los intervalos $\left[\frac{k-1}{m}, \frac{k}{m}\right]$ con $k = 1, \ldots, m$. Para eso, el árbol hace $m-1$ cortes verticales. En cada pedazo pone un escalón horizontal a la altura k/m: los puntos arriba del escalón los clasifica como clase 1 y los de abajo como clase 0. Mientras tanto, la diagonal sube $1/m$ en ese mismo ancho, de $\frac{k-1}{m}$ a $\frac{k}{m}$, así que la esquina de cada escalón toca la diagonal. La figura muestra el caso $m = 4$.



Escalera con $m = 4$ (azul), diagonal $x_2 = x_1$ (punteada) y región mal clasificada (rojo).

Dónde se equivoca. En cada pedazo, entre el escalón y la diagonal queda un triángulo rectángulo: son puntos que están arriba de la diagonal (clase 1) pero abajo del escalón, así que el árbol los clasifica como clase 0. Ahí el árbol clasifica mal. Su base mide $1/m$ (el ancho del pedazo) y su altura también $1/m$ (lo que sube la diagonal), así que su área es

$$\frac{1}{2} \cdot \frac{1}{m} \cdot \frac{1}{m} = \frac{1}{2m^2}.$$

El error total. Como hay m triángulos iguales, uno por pedazo, el área total mal clasificada es

$$m \cdot \frac{1}{2m^2} = \frac{1}{2m}.$$

Por ejemplo, con $m = 1$ (sin cortes, una sola hoja) el error es $1/2$, igual que adivinar, y con $m = 4$ es $1/8 = 12.5$ %. El error baja conforme se agregan escalones, pero solo de forma proporcional a $1/m$.

Cuántas hojas para equivocarse menos del 1 %. Queremos que el error sea menor a 0.01:

$$\frac{1}{2m} < 0.01 \iff 2m > 100 \iff m > 50.$$

Es decir, hacen falta al menos $m = 51$ escalones. Cada escalón necesita 2 hojas (la de arriba y la de abajo del escalón), así que el árbol necesita $2m = 102$ hojas, unas 100.

El árbol oblicuo. El árbol oblicuo puede cortar en diagonal: con la pregunta "¿$x_2 - x_1 \leq 0$?", que en la notación del enunciado es $w^\top x \leq t$ con $w = (-1, 1)$ y $t = 0$, separa las clases sin errores con 1 corte, usando 2 hojas. El árbol con cortes paralelos a los ejes necesita unas 100 hojas para equivocarse *todavía* en casi el 1 %.

(b) Un nodo a mano (5 pts)

Al nodo llegan los puntos de la clase 0: $(0,1)$, $(1,3)$, $(3,4)$, y de la clase 1: $(1,0)$, $(3,1)$, $(4,3)$.

Ningún corte paralelo a un eje los separa. Un corte vertical es una pregunta "¿$x_1 \le c$?" y separa sin errores solo si todos los valores de x_1 de una clase quedan de un lado de c y todos los de la otra clase del otro lado. Los valores de x_1 de la clase 0 son $\{0,1,3\}$ y los de la clase 1 son $\{1,3,4\}$: los dos rangos se enciman en $[1,3]$ (de hecho, $x_1 = 1$ y $x_1 = 3$ aparecen en las dos clases), así que para cualquier c queda algún punto del lado equivocado. Con x_2 pasa lo mismo: la clase 0 tiene $\{1,3,4\}$ y la clase 1 tiene $\{0,1,3\}$, que también se enciman en $[1,3]$. Por lo tanto, ningún corte horizontal ni vertical separa los seis puntos sin errores; el mejor de ellos se equivoca en dos puntos (por ejemplo, $x_1 \le 2$ deja $(1,0)$ a la izquierda y $(3,4)$ a la derecha).

Medias. Promediando cada coordenada dentro de cada clase:

$$m_0 = \left(\tfrac{0+1+3}{3}, \tfrac{1+3+4}{3}\right) = \left(\tfrac{4}{3}, \tfrac{8}{3}\right), \qquad m_1 = \left(\tfrac{1+3+4}{3}, \tfrac{0+1+3}{3}\right) = \left(\tfrac{8}{3}, \tfrac{4}{3}\right).$$

Matriz S_W. A cada punto le restamos la media de su clase, $(d_x, d_y) = x_i - m_c$, y sumamos las matrices $(x_i - m_c)(x_i - m_c)^\top = \begin{pmatrix} d_x^2 & d_x d_y \\ d_x d_y & d_y^2 \end{pmatrix}$ de los seis puntos:

Punto	Clase	(d_x, d_y)	d_x^2	$d_x d_y$	d_y^2
$(0,1)$	0	$(-\frac{4}{3}, -\frac{5}{3})$	$\frac{16}{9}$	$\frac{20}{9}$	$\frac{25}{9}$
$(1,3)$	0	$(-\frac{1}{3}, \frac{1}{3})$	$\frac{1}{9}$	$-\frac{1}{9}$	$\frac{1}{9}$
$(3,4)$	0	$(\frac{5}{3}, \frac{4}{3})$	$\frac{25}{9}$	$\frac{20}{9}$	$\frac{16}{9}$
$(1,0)$	1	$(-\frac{5}{3}, -\frac{4}{3})$	$\frac{25}{9}$	$\frac{20}{9}$	$\frac{16}{9}$
$(3,1)$	1	$(\frac{1}{3}, -\frac{1}{3})$	$\frac{1}{9}$	$-\frac{1}{9}$	$\frac{1}{9}$
$(4,3)$	1	$(\frac{4}{3}, \frac{5}{3})$	$\frac{16}{9}$	$\frac{20}{9}$	$\frac{25}{9}$
		Suma	$\frac{84}{9}$	$\frac{78}{9}$	$\frac{84}{9}$

Así,

$$S_W = \begin{pmatrix} \frac{28}{3} & \frac{26}{3} \\ \frac{26}{3} & \frac{28}{3} \end{pmatrix}.$$

Dirección de Fisher. El determinante de S_W es $\left(\frac{28}{3}\right)^2 - \left(\frac{26}{3}\right)^2 = \frac{784-676}{9} = 12 \neq 0$, así que S_W es invertible y, con la fórmula de la inversa de una matriz de 2×2,

$$S_W^{-1} = \frac{1}{12} \begin{pmatrix} \frac{28}{3} & -\frac{26}{3} \\ -\frac{26}{3} & \frac{28}{3} \end{pmatrix}.$$

Como $m_1 - m_0 = \left(\frac{4}{3}, -\frac{4}{3}\right)$, la dirección de Fisher (P1) es

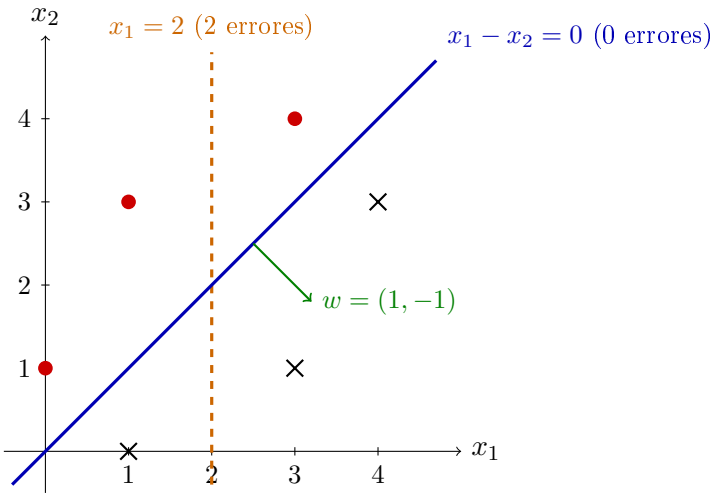
$$w = S_W^{-1}(m_1 - m_0) = \frac{1}{12} \begin{pmatrix} \frac{28}{3} \cdot \frac{4}{3} + \frac{26}{3} \cdot \frac{4}{3} \\ -\frac{26}{3} \cdot \frac{4}{3} - \frac{28}{3} \cdot \frac{4}{3} \end{pmatrix} = \frac{1}{12} \begin{pmatrix} \frac{216}{9} \\ -\frac{216}{9} \end{pmatrix} = \frac{1}{12} \begin{pmatrix} 24 \\ -24 \end{pmatrix} = \begin{pmatrix} 2 \\ -2 \end{pmatrix} \propto \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Solo importa la dirección de w (multiplicarlo por una constante positiva no cambia el orden de las proyecciones), así que usamos $w = (1, -1)$: la dirección perpendicular a la diagonal $x_2 = x_1$.

Proyección y umbral. Proyectar sobre w es calcular $w^\top x = x_1 - x_2$:

Punto	$(0,1)$	$(1,3)$	$(3,4)$	$(1,0)$	$(3,1)$	$(4,3)$
Clase	0	0	0	1	1	1
$x_1 - x_2$	-1	-2	-1	1	2	1

Todos los puntos de la clase 0 quedan en valores negativos (-2 y -1) y todos los de la clase 1 en valores positivos (1 y 2). Cualquier umbral entre -1 y 1 los separa; tomamos el punto medio, $t = 0$. El corte oblicuo "¿$x_1 - x_2 \leq 0$?" manda la clase 0 a un lado y la clase 1 al otro, sin errores.



Clase 0 (•), clase 1 (×), el mejor corte paralelo a un eje (naranja, punteado) y el corte oblicuo de Fisher (azul).

(c) Cuántos cortes hay que revisar (5 pts)

Cortes paralelos a un eje. Para una variable, ordenamos los n valores y solo importa entre qué par de valores consecutivos ponemos el umbral: hay $n - 1$ posiciones distintas. Con d variables hay $d(n-1)$ cortes distintos. Con $n = 1{,}000$:

$$d = 4: \ 4 \cdot 999 = 3{,}996 \text{ cortes}, \qquad d = 30: \ 30 \cdot 999 = 29{,}970 \text{ cortes}.$$

Cortes oblicuos. Por (P2), el número de maneras distintas de separar los n puntos con un hiperplano es $2\sum_{i=0}^{d} \binom{n-1}{i}$:

$$d = 4: \ 2\left[1 + 999 + \tbinom{999}{2} + \tbinom{999}{3} + \tbinom{999}{4}\right] \approx 8.3 \times 10^{10}, \qquad d = 30: \ 2\sum_{i=0}^{30} \tbinom{999}{i} \approx 4.9 \times 10^{57}.$$

Para $d = 4$ ya son unos 20 millones de veces más cortes que los paralelos, y para $d = 30$ el número es astronómico (para comparar, se estima que hay unos 10^{50} átomos en la Tierra). El número de cortes paralelos crece *linealmente* con d, mientras que el de cortes oblicuos crece como n^d.

¿Se pueden revisar todos los cortes oblicuos? No. Aun revisando mil millones de cortes por segundo, 8.3×10^{10} tomaría más de un minuto *por nodo* con $d = 4$, y 4.9×10^{57} es imposible. Por eso no se busca el mejor corte oblicuo por fuerza bruta: se usa una heurística que propone *una* dirección prometedora (aquí, la de Fisher) y solo se busca el umbral sobre ella, que de nuevo son $n - 1$ opciones.

Sesgo y varianza. Un corte más flexible tiene menos *sesgo*: puede acercarse más a la frontera real (por ejemplo, la diagonal del inciso (a) con un solo corte). Pero tiene más *varianza*: entre

más cortes posibles revisamos, más probable es que alguno separe bien los datos de entrenamiento *por casualidad*, ajustándose al ruido. Es el mismo fenómeno del Problema 1, inciso (b): al elegir el mejor entre m candidatos, el resultado se infla aproximadamente en proporción a $\sqrt{2\ln m}$, y aquí m crece como n^d, así que $\ln m$ crece como $d \ln n$. Con pocos datos por nodo (lo que pasa en los nodos profundos) y muchas dimensiones, el corte oblicuo puede sobreajustar mucho más que el paralelo.

(d) Qué no les afecta a cada uno (6 pts)

1. El árbol con cortes paralelos no cambia con una función estrictamente creciente. Sea f estrictamente creciente y supongamos que la aplicamos a la variable x_j. Como f es estrictamente creciente, $a < b \iff f(a) < f(b)$, así que f conserva el *orden* de los valores de x_j. Un corte "¿$x_j \le c$?" parte los puntos del nodo en dos grupos: los que tienen $x_j \le c$ y los demás. Después de transformar, el corte "¿$f(x_j) \le f(c)$?" produce exactamente los mismos dos grupos, porque $x_j \le c \iff f(x_j) \le f(c)$. Al revés, cualquier corte sobre $f(x_j)$ corresponde a uno sobre x_j. Por lo tanto, el conjunto de particiones posibles del nodo es el mismo antes y después de transformar. Como el índice de Gini depende solo de qué puntos quedan en cada grupo (no de los valores de x_j), el árbol elige la misma partición en cada nodo; a los hijos llegan los mismos puntos y, por inducción, todo el árbol hace las mismas particiones. Lo único que cambia es el número del umbral: si se toma el punto medio entre dos valores consecutivos $a < b$, antes era $\frac{a+b}{2}$ y después es $\frac{f(a)+f(b)}{2}$, que en general no es $f\big(\frac{a+b}{2}\big)$. Las otras variables no se ven afectadas.

2. La dirección de Fisher bajo $x' = Ax + c$. Sea A invertible. Las medias se transforman igual que los puntos, porque el promedio es lineal:

$$m'_k = \frac{1}{n_k} \sum_{i \in k} (Ax_i + c) = Am_k + c, \qquad k = 0, 1.$$

Entonces las desviaciones solo se multiplican por A (la traslación c se cancela): $x'_i - m'_k = A(x_i - m_k)$. Por lo tanto,

$$S'_W = \sum_k \sum_{i \in k} A(x_i - m_k)(x_i - m_k)^\top A^\top = A\, S_W\, A^\top, \qquad m'_1 - m'_0 = A(m_1 - m_0).$$

Usando $(ABC)^{-1} = C^{-1}B^{-1}A^{-1}$ y $(A^\top)^{-1} = A^{-\top}$:

$$w' = (S'_W)^{-1}(m'_1 - m'_0) = A^{-\top} S_W^{-1} A^{-1} A(m_1 - m_0) = A^{-\top} S_W^{-1}(m_1 - m_0) = A^{-\top} w.$$

Ahora proyectamos un punto transformado:

$$w'^\top x' = (A^{-\top} w)^\top (Ax + c) = w^\top A^{-1}(Ax + c) = w^\top x + w^\top A^{-1} c,$$

y $w^\top A^{-1} c$ es la misma constante para todos los puntos. Es decir, las proyecciones después de transformar son las de antes *desplazadas por una constante*. Desplazar todos los valores no cambia su orden, así que las particiones posibles sobre la proyección son las mismas, el Gini elige la misma, y solo el umbral se recorre: $t' = t + w^\top A^{-1} c$. Como a los hijos llegan los mismos puntos, por inducción todo el árbol hace las mismas particiones. Girar (A ortogonal), estirar (A diagonal con entradas no nulas) o trasladar (c) son casos de $x' = Ax + c$ con A invertible, así que los cortes del árbol oblicuo no cambian.

3. ¿Invariancias cruzadas?

- *El árbol con cortes paralelos no es invariante a una rotación.* La rotación mezcla las variables, y sus cortes solo pueden usar una variable a la vez. Por ejemplo, la frontera $x_1 > 0.2$ se resuelve con un solo corte; si giramos los datos 35°, la frontera se vuelve una línea inclinada y el árbol necesita una escalera de muchos escalones, como en el inciso (a).

- *El árbol oblicuo no es invariante a un logaritmo.* El logaritmo no es una transformación lineal, así que no es de la forma $Ax+c$ y el argumento anterior no aplica. En general, $w^\top \log(x)$ no es una función creciente de $w^\top x$, y las medias y S_W cambian de forma no lineal. Por ejemplo, la frontera $x_2 > x_1$ (un solo corte oblicuo) se convierte, con $x_1' = \log x_1$, en $x_2 > e^{x_1'}$, que es una curva: ya no la separa ningún hiperplano. La excepción es cuando el mejor corte usa una sola variable (w paralelo a un eje); entonces se comporta como un árbol normal en esa variable.

(e) Tu diseño (4 pts)

Decisiones (Tú decides).

- **Cuando S_W no es invertible**, el nodo usa solo cortes paralelos a un eje, como un árbol normal. Consideramos que S_W no es invertible si alguna variable es constante dentro del nodo o si el número de condición de S_W *normalizada* (cada entrada S_{ij} dividida entre $\sqrt{S_{ii}S_{jj}}$, como una matriz de correlación) es mayor que 10^{10}. Normalizamos porque el número de condición "crudo" depende de las unidades: en el vino, la varianza de `density` es del orden de 10^{-6} y la de `total sulfur dioxide` del orden de 10^3, y la matriz parecería casi singular sin serlo. Una matriz casi singular da direcciones inestables, no solo una exactamente singular. Esto pasa sobre todo en nodos con pocos datos o cuando alguna variable es constante en el nodo. Descartamos regularizar ($S_W + \lambda I$) porque introduce un parámetro más (λ) que habría que elegir, y descartamos la pseudo-inversa porque en matrices casi singulares da direcciones que dependen del ruido.
- **En cada nodo comparamos** el mejor corte oblicuo (dirección de Fisher con su mejor umbral) contra el mejor corte paralelo a un eje, y nos quedamos con el de menor Gini ponderado. Así el árbol nunca hace un corte peor que el de un árbol normal en ese nodo, y si la frontera real es paralela a un eje, la puede usar. Lo que perdemos es parte de la invariancia a rotaciones del inciso (d): cuando el árbol elige un corte paralelo, ese nodo sí depende de los ejes.

Pseudocódigo.

```
CONSTRUIR(X, y, profundidad):
    p <- proporción de la clase 1 en y
    si profundidad = max_profundidad, o todos los y son iguales,
       o hay menos de min_muestras puntos:
        regresar HOJA(p)

    candidatos <- []
    # Corte oblicuo (dirección de Fisher)
    si las dos clases tienen al menos 2 puntos:
        m0, m1 <- medias de cada clase
        S_W   <- suma de (x - m_c)(x - m_c)^T sobre todos los puntos
        si ninguna variable es constante y
           número de condición de S_W normalizada <= 1e10:
            w <- resolver S_W w = (m1 - m0)
            agregar MEJOR_UMBRAL(X w, y) con dirección w a candidatos
    # Cortes paralelos a cada eje
    para cada variable j:
        agregar MEJOR_UMBRAL(X[:, j], y) con dirección e_j a candidatos
```

```
    (w, t, gini) <- el candidato con menor Gini ponderado
    si gini >= Gini del nodo sin partir:
        regresar HOJA(p)              # ningún corte mejora
    izq <- puntos con w^T x <= t ;  der <- los demás
    regresar NODO(w, t, CONSTRUIR(izq, profundidad + 1),
                        CONSTRUIR(der, profundidad + 1))

MEJOR_UMBRAL(z, y):           # z: un número por punto
    ordenar z (y sus y)
    para cada par de valores consecutivos distintos z_(i) < z_(i+1):
        t <- (z_(i) + z_(i+1)) / 2
        G <- (n_izq/n) Gini(izq) + (n_der/n) Gini(der),
             con Gini(grupo) = 1 - p0^2 - p1^2
    regresar el t con menor G (y su G)

PREDECIR_PROBA(x): bajar por el árbol eligiendo izquierda si w^T x <= t;
                   regresar el p de la hoja (proporción de clase 1)
PREDECIR(x):       1 si PREDECIR_PROBA(x) >= 1/2, si no 0
```

Cuándo detenerse: cuando se llega a la profundidad máxima (que se elige con validación cruzada en la parte práctica), cuando el nodo es puro, cuando quedan menos de `min_muestras` puntos, o cuando ningún corte reduce el Gini. **Qué predice cada hoja:** la proporción de puntos de clase 1 que llegaron a ella en el entrenamiento; esa es la probabilidad de `predict_proba`, y `predict` da la clase mayoritaria.

2.2. Parte práctica

El código, las tablas y las gráficas están en `problema-2/problema_2.ipynb` y en el `README` de esa carpeta. En resumen: el árbol oblicuo encuentra en los seis puntos de (b) exactamente $w = (2, -2)$; con los datos girados 35° tiene 100 % de exactitud desde profundidad 1, mientras que el árbol de `scikit-learn` empieza en 82 %; y en el vino tinto empata en AUC-ROC con el árbol de `scikit-learn` (0.837 contra 0.834) pero pierde en AUC-PR (0.435 contra 0.503) por sobreajuste, algo que el *bagging* corrige (0.737, al nivel del bosque aleatorio, 0.744).

2.3. Bitácora de decisiones — Problema 2

Decisión 1: Qué hacer cuando S_W no es invertible (Tú decides, teoría (e))

Decisión. Si S_W no es invertible (alguna variable constante en el nodo o número de condición de S_W normalizada mayor que 10^{10}), el nodo usa solo cortes paralelos a un eje.

Alternativas descartadas. Regularizar con $S_W + \lambda I$ y usar la pseudo-inversa.

Por qué. Regularizar agrega un hiperparámetro (λ) que habría que elegir con validación cruzada, y la pseudo-inversa da direcciones que dependen del ruido cuando la matriz está casi singular. Como de todas formas comparamos contra los cortes paralelos (decisión 2), la opción más simple es que el corte oblicuo no participe en ese nodo.

Evidencia. En el vino, las variables tienen escalas de 10^{-6} a 10^{3}; con el número de condición sin normalizar, casi ningún nodo habría usado cortes oblicuos. Con la normalización, el árbol de profundidad 4 entrenado con todos los vinos usa cortes oblicuos en 15 de sus 15 nodos internos.

Qué perdemos. En nodos pequeños o con variables casi constantes, el árbol se comporta como uno normal y pierde la invariancia a rotaciones en ese nodo.

Decisión 2: Comparar el corte oblicuo contra el paralelo (Tú decides, teoría (e))

Decisión. En cada nodo calculamos el mejor corte oblicuo (dirección de Fisher) y el mejor corte paralelo a cada eje, y nos quedamos con el de menor Gini ponderado.

Alternativa descartada. Usar siempre el corte oblicuo.

Por qué. Así el árbol nunca hace en un nodo un corte peor (en Gini) que el de un árbol normal, y cuando la frontera es paralela a un eje la puede usar.

Evidencia. Con los datos de la práctica (b) transformados con $x_1 \mapsto e^{3x_1}$, el árbol oblicuo mantuvo 100 % de exactitud porque eligió el corte paralelo, que es invariante a funciones crecientes. Con los datos girados, eligió el oblicuo y también tuvo 100 %.

Qué perdemos. Parte de la invariancia a rotaciones (en los nodos donde gana el corte paralelo) y un poco más de cálculo por nodo ($d + 1$ búsquedas de umbral en vez de 1).

Decisión 3: Métrica de comparación (Tú decides, práctica (c))

Decisión. AUC-PR (*average precision*) como métrica principal y AUC-ROC como complemento.

Alternativas descartadas. Exactitud, exactitud balanceada y F1.

Por qué. Solo el 13.6 % de los vinos tiene calidad ≥ 7: decir "malo" a todos da 86 % de exactitud. La exactitud balanceada y F1 dependen del umbral de 0.5, y los árboles poco profundos casi nunca predicen la clase minoritaria con ese umbral. AUC-PR evalúa qué tan bien quedan los vinos buenos hasta arriba del orden, y su valor de azar es la proporción de positivos (0.136), así que es fácil de interpretar.

Evidencia. La diferencia entre el árbol oblicuo y el de `scikit-learn` solo se ve con AUC-PR (0.435 contra 0.503); con AUC-ROC empatan (0.837 contra 0.834). Con clases desbalanceadas, AUC-ROC puede esconder diferencias en la parte del orden que importa.

Qué perdemos. AUC-PR es más ruidosa entre pliegues (desviación de hasta 0.076) y es menos conocida que la exactitud para explicar los resultados.

Decisión 4: Elegir la profundidad con validación cruzada anidada (Tú decides, práctica (c))

Decisión. Cinco pliegues externos para evaluar; dentro de cada uno, otra validación cruzada de cinco pliegues elige la profundidad (de 1 a 10) solo con el entrenamiento.

Alternativa descartada. Una sola validación cruzada por profundidad, reportando la mejor.

Por qué. Reportar el mejor resultado entre 10 profundidades con los mismos datos infla el número (el sesgo del inciso (b) del Problema 1).

Evidencia. Las profundidades elegidas cambian entre pliegues (de 2 a 5 para el árbol oblicuo), lo que muestra que la elección es ruidosa y que reportar el máximo de una sola curva sería optimista. En la curva del inciso (d), el máximo de validación del árbol con cortes paralelos es 0.529, mayor que el 0.503 de la validación anidada del árbol de `scikit-learn`.

Qué perdemos. Tiempo de cálculo (unas 250 veces más entrenamientos por modelo) y que no hay "una" profundidad elegida, sino una por pliegue.

Decisión 5: Cómo poner a prueba por qué perdió el árbol oblicuo (práctica (d))

Decisión. Comparar el AUC-PR de entrenamiento y validación por profundidad del árbol oblicuo contra el *mismo código* con `oblicuo=False`, y contra el árbol oblicuo con $\log(1 + x)$.

Alternativa descartada. Comparar directamente contra el árbol de `scikit-learn`.

Por qué. Con el mismo código, la única diferencia es el tipo de corte, así que cualquier diferencia se debe a los cortes oblicuos y no a detalles de implementación. El logaritmo pone a prueba la hipótesis alternativa (valores extremos).

Evidencia. El árbol oblicuo tiene mayor AUC-PR en entrenamiento pero menor en validación desde profundidad 3 (por ejemplo, 0.454 contra 0.529 en profundidad 5): sobreajuste. El logaritmo no lo mejora. El *bagging* de árboles oblicuos, que reduce la varianza, sube el AUC-PR de 0.435 a 0.737. Además, nuestro árbol con `oblicuo=False` da casi lo mismo que el de `scikit-learn`, lo que valida la implementación.

Qué perdemos. El experimento usa una sola validación cruzada por profundidad (no anidada), así que sus números absolutos son un poco optimistas; lo que importa es la comparación entre variantes, que usan los mismos pliegues.

Declaración de uso de asistentes de IA

Herramientas utilizadas

- **Gemini** (Google), con el modelo Gemini 4 Argon, dentro de Antigravity IDE (fork de VS Code). Tiene acceso a los archivos del repositorio y puede ejecutar código.
- **Claude Code** (Anthropic), con el modelo Claude Opus 5.5, dentro de Antigravity IDE (fork de VS Code). Tiene acceso a los archivos del repositorio y puede ejecutar código.

Problema 1

Parte teórica. Usé Gemini como tutor. Como estuve ausente en varias clases de este tema, le pedí que me explicara a qué se refería cada pregunta y cómo podía abordarla, *sin darme la respuesta*. Con esas explicaciones hice yo los cálculos: la fracción de positivos después de repetirlos (48.12 %), la probabilidad de que un positivo de prueba tenga una copia en el entrenamiento ($1 - 0.2^6$), la exactitud medida en función de p, los sesgos de 3.50 % (mi caso) y 0.54 % (la consultora) en el inciso (b), y la correlación máxima de ≈ 0.58 en el inciso (c). Gemini revisó mis operaciones, me dio una plantilla de LaTeX para el reporte y pasó en limpio, en prosa, las explicaciones que yo le di en la conversación.

En el inciso (d), el argumento sobre `duration` sale de mi experiencia trabajando en un centro de llamadas (vendiendo paquetes vacacionales para Air Canada): una llamada larga no garantiza una venta, pero una muy corta casi seguro no la tiene. A partir de eso propuse la regla de dos etapas (antes y después de llamar) e identifiqué que `campaign`, `day` y `month` tienen el mismo problema. En el inciso (e), la idea de que los meses de la crisis de 2008 contaminan el modelo y la propuesta de una ventana deslizante también son mías.

Parte práctica. Con Gemini armé el notebook inicial: modifiqué yo el código de la consultora con mis parámetros (inciso (a)), escribí la tabla de problemas (inciso (b)) y las celdas de las variantes del inciso (c). Gemini me ayudó a depurar los errores que me salieron.

Después usé Claude Code para revisar y continuar el trabajo:

- **Repositorio:** corrigió el nombre del `.gitignore` y se encargó de los *commits* y de subirlos a GitHub.
- **Inciso (c):** detectó que varias de mis celdas no corregían un solo problema a la vez (por ejemplo, la de copias quitaba el sobremuestreo por completo y la temporal corregía dos cosas). Me guió celda por celda para corregirlas: yo hice los cambios y corrí cada celda, comparando con los valores esperados. La celda de "todos corregidos" me la dio completa, con la explicación de cada bloque. También agregó los títulos de las celdas y la tabla resumen.
- **Inciso (d):** programó la versión honesta (el `Pipeline`, la validación cruzada temporal, el intervalo por *bootstrap* y la curva de ganancia). También programó la prueba de mi hipótesis de la ventana deslizante, con un *bootstrap* pareado, y encontró que la tasa de contratación cambia mucho con el tiempo.
- **Inciso (e):** programó la simulación del ruido y su gráfica, y me explicó por qué la correlación medida (0.516) es menor que la que predice (P1).

Redacción. Los comentarios de los incisos (c) y (d) del notebook y la carta al banco parten de borradores míos, que Claude corrigió y completó. En la carta reordenó los errores para que coincidieran con mis resultados y agregó recomendaciones. El comentario del inciso (e), la bitácora de decisiones y el `README` del problema 1 los redactó Claude a partir de las decisiones que tomamos en la conversación.

Decisiones propias. Las decisiones de fondo fueron mías: usar el modelo *antes* de llamar y, por lo tanto, quitar `duration`, `campaign`, `day` y `month`; respetar el orden cronológico de los datos; la hipótesis de la ventana deslizante; y usar regresión logística en el experimento del ruido. Claude propuso la métrica de reporte (precisión en el 10 % superior) y comparar `class_weight="balanced"` contra no balancear; yo las acepté después de que me explicara por qué tenían sentido para el uso del banco.

Problema 2

Parte teórica. Claude me explicó primero la idea del problema sin ecuaciones (cortes paralelos, la escalera, el corte oblicuo y la dirección de Fisher). En los incisos (a) y (b) me guió con pasos con huecos que yo llené: hice el cálculo del área de la escalera, el despeje de m y el conteo de hojas, y en (b) calculé a mano las medias, la tabla de S_W, el determinante, la inversa, la dirección w y las proyecciones; Claude corrigió algunos errores de álgebra y de signos, pasó mis respuestas a LaTeX e hizo los dibujos. Los incisos (c), (d) y (e) los redactó Claude por falta de tiempo; yo tomé las dos decisiones del inciso (e) (usar solo cortes paralelos cuando S_W no es invertible y comparar el corte oblicuo contra el paralelo en cada nodo) entre las opciones que me propuso.

Parte práctica. Claude programó el árbol oblicuo, su prueba automática y todos los experimentos del notebook, y redactó los comentarios y el `README`. Durante el trabajo encontró y corrigió un error propio: el orden de la herencia (`ClassifierMixin` debe ir antes que `BaseEstimator`), que hacía fallar en silencio la métrica AUC. Yo elegí la métrica de comparación (AUC-PR y AUC-ROC) y la validación cruzada anidada para elegir la profundidad.

Redacción. La bitácora del Problema 2 la redactó Claude a partir de las decisiones anteriores.

Cómo verifiqué lo que hizo la IA

Corrí yo mismo cada celda del notebook y comparé sus resultados con los valores esperados. Cuando no entendía algo (por ejemplo, el experimento del ruido o por qué balancear no cambia el resultado), pedí que me lo explicaran con un ejemplo antes de aceptarlo. También revisé que los resultados prácticos fueran coherentes con la parte teórica, y cuando no lo eran (las copias en el inciso (c) y la correlación en el inciso (e)), pedí la explicación de la diferencia.

Esta declaración fue redactada con ayuda de Claude, a partir del historial de mis sesiones con ambos asistentes.